One-Pager — Expert-Learning Organizational Reasoning Engine

Problem

Teams rely on a few experts to answer everyday questions like "why was the release delayed?" or "who owns this service?". AI assistants can fetch facts, but they miss the root cause and the context an expert brings, and they don't get better when they are corrected.

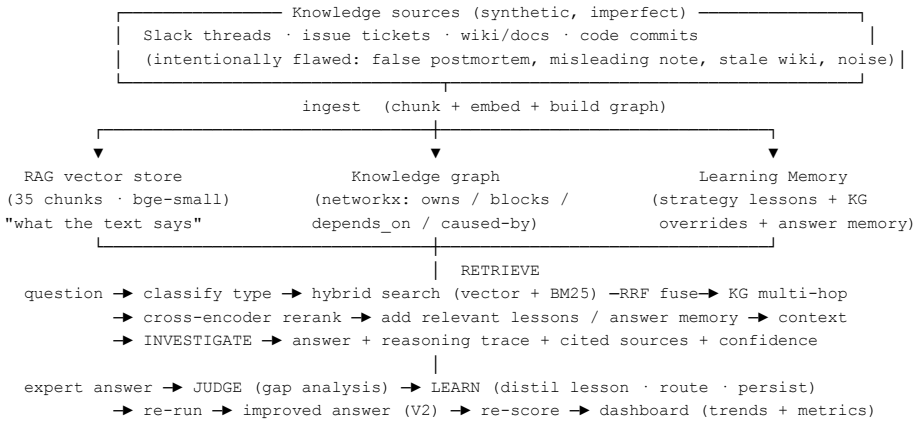
What it does

1. Investigates synthetic knowledge sources (Slack, tickets, wiki, commits) and writes an answer with a reasoning trace, citations, and a confidence score.
2. Captures the human expert's answer.
3. Runs gap analysis, creates a learning event, and saves it.
4. Re-runs to produce a better V2, and reuses what it learned on new questions of the same type.

How it learns (no fine-tuning)

The same model answers both times; only what it retrieves changes. The first answer works from the (imperfect) sources alone. After the expert weighs in, three things are saved as data: an answer memory keyed to the exact question (so a re-run of that question matches the expert), strategy lessons tagged by question type (so the approach carries to new questions), and knowledge-graph overrides for factual fixes. On the re-run, retrieval pulls these back into context and the answer improves. The model's weights never change; recall is the memory.

Architecture



Pipeline:

1. Ingest: each source becomes one self-contained, tagged chunk (embedded into the RAG store); structured fields build the knowledge graph (owns, blocks, depends_on, caused-by).
2. Retrieve: classify the question type, run hybrid search (vector + BM25) fused with RRF, add knowledge-graph multi-hop to find linked-but-buried evidence, then rerank to the best few; relevant lessons and any exact-question answer memory are added.
3. Investigate: the LLM reasons over that context and returns an answer with a reasoning trace, cited source IDs, and a confidence score. Cited IDs are checked against what was retrieved, so it cannot invent a source. If evidence is thin, it says so instead of guessing.
4. Judge: gap analysis against the expert answer (similarity, root-cause match, evidence coverage, and what was missed).
5. Learn: write a type-tagged strategy lesson, store the expert answer and any factual fix, then re-run to show V2.

Why three stores: RAG covers "what do the docs say", the knowledge graph covers "how is it connected" (deterministic, no hallucination), and the Learning Memory covers "what did we learn from experts". No single store is enough; together they cover text, structure, and learning.

Results

Across 10 complex, multi-source questions:

Success metric	Target	Result

AI answer similarity to expert (V2)	≥ 75%	88% (pass)
Root-cause identification (V2)	≥ 70%	94% (pass)
Evidence coverage (V2)	≥ 80%	95% (pass)
Response time	< 15 s	~3.6 s (pass)
Test-scenario coverage	100%	10/10 (pass)
Learning improvement (V1 → V2)	≥ 20%	+9% (V1 79% → V2 88%)

Five of six criteria pass. Because the same model answers both passes, the first answer is already fairly strong (around 79%), so the average improvement is +9%, under the 20% target. The biggest gains show up where the data is misleading: the wrong INC-9 postmortem ("load spike") and the stale owner wiki push the first answer wrong (as low as 66%), and the expert correction lifts V2 to around 90% (+13% to +23% on those questions). Numbers are reported per question, not as a single figure.

Assumptions

Synthetic data only. Human answers are the ground truth. Any LLM works (this uses the Groq free tier, with local Ollama and an offline stub as alternatives). Runs on Linux and macOS.

What I'd build next

A production version that:

- ingests a live Slack feed and answers incoming questions in real time;
- checks each answer's confidence, and when it is low, routes to a human to capture the expert answer;
- learns from that answer, so the next similar question is answered on its own.